

Практическое занятие №4: Полный цикл предобработки данных

Цель занятия

Научиться применять на практике все изученные приёмы подготовки данных: от загрузки «сырого» датасета до получения чистой, структурированной таблицы, готовой к анализу или моделированию. Вы пройдёте полный цикл очистки, преобразования и обогащения данных на примере реального набора данных о пассажирах «Титаника».

Задачи

1. Загрузить данные из внешнего источника (CSV-файл).
2. Провести первичный анализ структуры и качества данных.
3. Обработать пропущенные значения различными методами.
4. Выявить и удалить дубликаты.
5. Обнаружить и обработать выбросы.
6. Создать новые признаки на основе существующих.
7. Отфильтровать ненужные данные и сохранить результат.

Теоретические основы (краткое повторение)

1. Этапы предобработки данных

- **Загрузка данных:** `pd.read_csv()`, `pd.read_excel()` и т.д.
- **Первичный осмотр:** `.info()`, `.describe()`, `.head()`, `.shape`, проверка типов.
- **Обработка пропусков:**
 - удаление строк/столбцов с пропусками (`.dropna()`)
 - заполнение константой, средним/медианой/модой (`.fillna()`)
 - интерполяция (для временных рядов)
- **Обработка дубликатов:** поиск (`.duplicated()`), удаление (`.drop_duplicates()`).
- **Обработка выбросов:**
 - визуализация (ящик с усами, гистограмма)
 - статистические методы (z-оценка, межквартильный размах)
 - замена или удаление
- **Преобразование данных:**
 - создание новых признаков (например, длина строки, день недели)
 - группировка и агрегирование
 - фильтрация строк
- **Сохранение результата:** `.to_csv()`, `.to_excel()`.

2. Ключевые методы pandas

- `df.info()` – информация о DataFrame (количество непустых значений, типы).
- `df.isnull().sum()` – количество пропусков в каждом столбце.
- `df.describe(include='all')` – описательная статистика для всех столбцов.
- `df.duplicated()` – поиск дублированных строк.
- `df.drop_duplicates()` – удаление дубликатов.

- `df.fillna(value)` – заполнение пропусков.
- `df[condition]` – фильтрация строк.
- `df.apply(func)` – применение функции к каждому элементу столбца.
- `df['new_col'] = ...` – создание нового столбца.
- `df.groupby(by).agg(...)` – группировка и агрегация.

Подробный пример: Датасет «Титаник»

Шаг 1. Загрузка данных

Датасет «Титаник» – классический набор данных, содержащий информацию о пассажирах (выжил/не выжил, класс, возраст, пол и т.д.). Мы будем использовать файл `titanic.csv`, который можно скачать, например, с [Kaggle](#) или использовать встроенный в `seaborn`.

В учебных целях загрузим его прямо из библиотеки `seaborn` (она часто уже установлена вместе с `Anaconda`). Если `seaborn` нет, установите: `!pip install seaborn`.

```
import pandas as pd
import numpy as np
import seaborn as sns

# Загружаем датасет titanic из библиотеки seaborn
df = sns.load_dataset('titanic')
print("Размер исходных данных:", df.shape)
df.head()
```

Шаг 2. Первичный осмотр и выявление проблем

Выведем общую информацию о данных:

```
df.info()
```

Результат:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   survived        891 non-null    int64
 1   pclass          891 non-null    int64
 2   sex             891 non-null    object
 3   age            714 non-null    float64
 4   sibsp          891 non-null    int64
 5   parch          891 non-null    int64
 6   fare           891 non-null    float64
 7   embarked       889 non-null    object
 8   class          891 non-null    category
 9   who            891 non-null    object
10  adult_male     891 non-null    bool
11  deck          203 non-null    category
12  embark_town    889 non-null    object
13  alive         891 non-null    object
14  alone         891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.6+ KB
```

Видим пропуски в столбцах age, embarked, deck, embark_town. Особенно много пропусков в deck — только 203 записи из 891.

Шаг 3. Анализ пропусков

```
# Количество пропусков в каждом столбце
print(df.isnull().sum())
text
survived          0
pclass            0
sex              0
age             177
sibsp            0
parch            0
fare             0
embarked          2
class            0
who              0
adult_male        0
deck            688
embark_town        2
alive            0
alone            0
dtype: int64
```

Вывод: много пропусков в deck (77%), меньше в age (около 20%), совсем немного в embarked и embark_town.

Шаг 4. Обработка пропусков

4.1. Столбец deck

Поскольку пропусков слишком много, и эта информация, скорее всего, не будет критичной для простого анализа, можно просто удалить этот столбец.

```
df.drop(columns=['deck'], inplace=True)
```

4.2. Столбец embarked и embark_town

Эти столбцы дублируют друг друга (embarked — код порта, embark_town — название). Всего 2 пропуска. Можно удалить строки с пропусками или заполнить наиболее частым значением (модой). Удалим строки с пропусками, так как их мало.

```
df.dropna(subset=['embarked'], inplace=True)
# после удаления проверяем размер
print(df.shape)
```

4.3. Столбец age

Возраст — важный признак. Удалять 177 строк нежелательно. Можно заполнить медианным возрастом по группам, например, по классу и полу.

Сначала посмотрим распределение возраста:

```
df['age'].describe()
```

Среднее ~30, медиана тоже около 30.

Заполним пропуски медианным возрастом для каждого сочетания pclass и sex:

```
df['age'] = df.groupby(['pclass', 'sex'])['age'].transform(lambda x:
x.fillna(x.median()))
```

Проверим, остались ли пропуски:

```
df.isnull().sum()
```

Возраст должен быть полностью заполнен.

Шаг 5. Обработка дубликатов

Проверим, есть ли полностью дублированные строки:

```
duplicates = df.duplicated()
print("Количество дубликатов:", duplicates.sum())
```

Если есть, удалим их:

```
df.drop_duplicates(inplace=True)
```

Шаг 6. Обработка выбросов

Посмотрим на столбец fare (цена билета) с помощью ящика с усами:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
df['fare'].plot(kind='box')
plt.title('Boxplot of Fare')

plt.subplot(1,2,2)
df['fare'].hist(bins=50)
plt.title('Histogram of Fare')
plt.show()
```

Видим, что есть несколько очень больших значений (более 500). Возможно, это выбросы. Определим верхнюю границу методом межквартильного размаха:

```
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
upper_bound = Q3 + 1.5 * IQR
lower_bound = Q1 - 1.5 * IQR

outliers = df[(df['fare'] > upper_bound) | (df['fare'] < lower_bound)]
print(f"Количество выбросов: {len(outliers)}")
```

Решение: для анализа можно либо удалить выбросы, либо заменить их на граничные значения (винзоризация). В учебных целях оставим как есть, но отметим, что при построении моделей их стоит обработать.

Шаг 7. Создание новых признаков

Создадим несколько полезных признаков:

- `family_size` – размер семьи (`sibsp + parch + 1`)
- `is_alone` – был ли пассажир один (если `family_size == 1`)
- `age_group` – возрастная группа (ребёнок, взрослый, пожилой)

```
df['family_size'] = df['sibsp'] + df['parch'] + 1
df['is_alone'] = (df['family_size'] == 1).astype(int)
```

```
def age_category(age):
    if age < 18:
        return 'child'
    elif age < 60:
        return 'adult'
    else:
        return 'elderly'
```

```
df['age_group'] = df['age'].apply(age_category)
```

Шаг 8. Фильтрация и группировка

Отфильтруем данные, оставив только пассажиров первого класса:

```
df_first = df[df['pclass'] == 1]
```

Посчитаем средний возраст и цену билета по полу для первого класса:

```
python
df_first.groupby('sex')[['age', 'fare']].mean()
```

Шаг 9. Сохранение результата

Сохраним очищенный и обогащённый датасет в новый CSV-файл:

```
df.to_csv('titanic_cleaned.csv', index=False)
```

Шаг 10. Итоговый код (весь процесс)

Приведём весь код компактно, чтобы его можно было скопировать и выполнить.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Загрузка
df = sns.load_dataset('titanic')

# Удаление столбца с большим числом пропусков
df.drop(columns=['deck'], inplace=True)

# Удаление строк с пропусками в embarked
df.dropna(subset=['embarked'], inplace=True)

# Заполнение возраста медианой по группе
```

```

df['age'] = df.groupby(['pclass', 'sex'])['age'].transform(lambda x:
x.fillna(x.median()))

# Удаление дубликатов
df.drop_duplicates(inplace=True)

# Создание новых признаков
df['family_size'] = df['sibsp'] + df['parch'] + 1
df['is_alone'] = (df['family_size'] == 1).astype(int)
df['age_group'] = df['age'].apply(lambda age: 'child' if age < 18 else ('adult'
if age < 60 else 'elderly'))

# Сохранение
df.to_csv('titanic_cleaned.csv', index=False)

print("Предобработка завершена. Размер итогового датасета:", df.shape)

```

Варианты для самостоятельного выполнения (10 вариантов)

Для каждого варианта предлагается свой датасет и специфические задачи. Студент должен загрузить данные, выполнить полный цикл предобработки и сохранить результат. Данные можно взять из открытых источников (например, из репозитория UCI или seaborn). Ниже приведены описания датасетов и конкретные указания.

Вариант 1: Датасет «Flights» (пассажиропоток авиакомпаний)

- **Источник:** `sns.load_dataset('flights')`
- **Описание:** Ежемесячное количество пассажиров авиакомпании за 1949–1960 годы.
- **Задачи:**
 1. Проверить наличие пропусков (если есть – заполнить линейной интерполяцией).
 2. Создать столбец `year_month` как строку вида "1949-01".
 3. Добавить столбец `season` (зима, весна, лето, осень) на основе месяца.
 4. Отфильтровать данные только за летние месяцы (июнь-август).
 5. Построить график зависимости пассажиров от времени.

Вариант 2: Датасет «Iris» (цветки ириса)

- **Источник:** `sns.load_dataset('iris')`
- **Описание:** Длина и ширина чашелистика и лепестка для трёх видов ириса.
- **Задачи:**
 1. Проверить данные на наличие пропусков и дубликатов.
 2. Создать новый признак `petal_area` (произведение длины и ширины лепестка).
 3. Создать признак `sepal_area` (аналогично).
 4. Удалить все строки, где `petal_length` меньше 1.5 (если такие есть).
 5. Сгруппировать по виду и вычислить средние значения всех признаков.

Вариант 3: Датасет «Taxi» (поездки на такси в Нью-Йорке)

- **Источник:** `sns.load_dataset('taxi')`
- **Описание:** Данные о поездках такси (дата, расстояние, стоимость, чаевые и т.д.).
- **Задачи:**
 1. Привести столбец `pickup` и `dropoff` к типу `datetime`.
 2. Создать признаки: час пик (6-9, 17-20), день недели, выходной ли день.
 3. Найти строки с нулевым расстоянием и решить, что с ними делать.
 4. Заполнить пропуски в `dropoff_zone` значением "Unknown".
 5. Посчитать среднюю стоимость поездки по зонам посадки.

Вариант 4: Датасет «Diamonds» (характеристики алмазов)

- **Источник:** `sns.load_dataset('diamonds')`
- **Описание:** Цена и характеристики алмазов (карат, огранка, цвет, прозрачность).
- **Задачи:**
 1. Проверить наличие пропусков.
 2. Удалить дубликаты, если есть.
 3. Создать признак `volume = x*y*z` (если есть такие столбцы; в датасете есть `x`, `y`, `z` – габариты).
 4. Отфильтровать алмазы с ценой выше 10000 и каратами больше 1.
 5. Построить зависимость цены от карата для разных типов огранки.

Вариант 5: Датасет «Tips» (чаевые в ресторане)

- **Источник:** `sns.load_dataset('tips')`
- **Описание:** Сумма счёта, чаевые, пол, день, время, размер компании.
- **Задачи:**
 1. Проверить на пропуски (их нет).
 2. Добавить столбец `tip_percentage = tip/total_bill * 100`.
 3. Создать признак `group_size_category` (один, пара, компания).
 4. Отфильтровать строки, где чаевые превышают 25% от счёта (возможно, ошибка).
 5. Найти средний процент чаевых по дням недели и времени (обед/ужин).

Вариант 6: Датасет «Exercise» (физические упражнения)

- **Источник:** `sns.load_dataset('exercise')`
- **Описание:** Пульс, время, тип диеты и вид упражнений.
- **Задачи:**
 1. Определить тип данных каждого столбца.
 2. Преобразовать столбец `time` в категориальный (если он не категория).
 3. Найти и обработать пропуски (если есть).
 4. Создать признак `pulse_change = пульс после - пульс до` (если есть такие данные).
 5. Сгруппировать по диете и виду упражнений, вычислить средний пульс.

Вариант 7: Датасет «Penguins» (пингвины)

- **Источник:** `sns.load_dataset('penguins')`
- **Описание:** Размеры клюва, ласт, масса тела для трёх видов пингвинов.

- **Задачи:**
 1. Обработать пропуски (в этом датасете они есть) – заполнить медианой по виду.
 2. Удалить строки, где пропущены вид или остров.
 3. Создать признак `bill_area` (длина * глубина клюва).
 4. Отфильтровать пингвинов с массой тела выше среднего.
 5. Построить гистограммы распределения массы по видам.

Вариант 8: Датасет «Planets» (экзопланеты)

- **Источник:** `sns.load_dataset('planets')`
- **Описание:** Масса, период, год открытия экзопланет, метод обнаружения.
- **Задачи:**
 1. Посмотреть количество пропусков в каждом столбце.
 2. Заполнить пропуски в столбце `mass` медианой по методу обнаружения.
 3. Удалить строки, где `distance` пропущена (если таких немного).
 4. Создать признак `discovery_decade` (например, 1990-е, 2000-е и т.д.).
 5. Найти среднюю массу планет, открытых разными методами, за последние 20 лет.

Вариант 9: Датасет «MPG» (расход топлива автомобилей)

- **Источник:** `sns.load_dataset('mpg')`
- **Описание:** Расход топлива, мощность, вес, ускорение автомобилей.
- **Задачи:**
 1. Обработать пропуски в столбце `horsepower` (можно заполнить средним).
 2. Удалить строки с пропущенным `origin`.
 3. Создать признак `power_to_weight = horsepower / weight`.
 4. Отфильтровать автомобили с расходом более 30 миль на галлон.
 5. Сравнить средний расход для автомобилей разных стран происхождения.

Вариант 10: Датасет «Attention» (внимание и время реакции)

- **Источник:** `sns.load_dataset('attention')`
- **Описание:** Время реакции испытуемых в зависимости от внимания и типа задания.
- **Задачи:**
 1. Привести данные к широкому формату: строки – испытуемые, столбцы – время реакции при разных условиях (подсказка: `pivot`).
 2. Удалить дубликаты.
 3. Проверить наличие выбросов во времени реакции (использовать IQR).
 4. Заменить выбросы на границы IQR.
 5. Построить ящики с усами для разных уровней внимания.

Контрольные вопросы для самопроверки

1. Какие основные этапы входят в полный цикл предобработки данных?
2. Как с помощью pandas определить количество пропусков в каждом столбце?
3. В каких случаях лучше удалить строки с пропусками, а в каких – заполнить их?
4. Какими способами можно заполнить пропуски в числовых данных? В категориальных?
5. Как найти и удалить дубликаты в DataFrame?
6. Что такое выбросы? Как их можно обнаружить статистическими методами?
7. Приведите пример создания нового признака на основе существующих столбцов.
8. Как выполнить группировку данных и вычислить средние значения по группам?
9. Для чего нужно сохранять очищенные данные в отдельный файл?
10. Почему важно проверять типы данных столбцов после загрузки?

Дополнительные источники информации и литература

1. Документация pandas по обработке пропусков:
https://pandas.pydata.org/docs/user_guide/missing_data.html
2. Документация по работе с дубликатами:
<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.duplicated.html>
3. Статья "Handling outliers in pandas" (на английском):
<https://towardsdatascience.com/ways-to-detect-and-remove-the-outliers-404d16608dba>
4. Уэс Маккинни "Python для анализа данных", глава 7 – очистка и подготовка данных.
5. Kaggle Learn – "Data Cleaning" course:
<https://www.kaggle.com/learn/data-cleaning>
6. Примеры работы с реальными датасетами на русском:
<https://habr.com/ru/post/489856/>

Критерии оценивания выполнения самостоятельной работы

- (10 баллов) Загрузка данных и первичный осмотр (info, describe, head).
- (20 баллов) Обработка пропусков (обоснованный выбор метода).
- (10 баллов) Обработка дубликатов.
- (20 баллов) Обработка выбросов (если требуется в задании).
- (20 баллов) Создание новых признаков согласно заданию.
- (10 баллов) Фильтрация или группировка по заданию.
- (10 баллов) Сохранение результата и вывод итоговой информации.

Максимум: 100 баллов.